

Bachelor-Thesis

zur Erlangung des akademischen Grades

Bachelor of Science (B. Sc.)

an der Hochschule für Technik und Wirtschaft des Saarlandes

im Studiengang Praktische Informatik

der Fakultät für Ingenieurwissenschaften

Netzwerk-Monitoring Dashboard mit Automatisierung

vorgelegt von

Xudong Zhang

betreut und begutachtet von

Prof. Dr.-Ing. André Miede

Andreas Schaffhauser

Saarbrücken, den 4. August 2026

Selbständigkeitserklärung

Ich versichere, dass ich die vorliegende Arbeit (bei einer Gruppenarbeit: den entsprechend gekennzeichneten Anteil der Arbeit) selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Ich erkläre hiermit weiterhin, dass die vorgelegte Arbeit zuvor weder von mir noch von einer anderen Person an dieser oder einer anderen Hochschule eingereicht wurde.

Darüber hinaus ist mir bekannt, dass die Unrichtigkeit dieser Erklärung eine Benotung der Arbeit mit der Note „nicht ausreichend“ zur Folge hat und einen Ausschluss von der Erbringung weiterer Prüfungsleistungen zur Folge haben kann.

Saarbrücken, den 4. August 2026

Xudong Zhang

Zusammenfassung

Kurze Zusammenfassung des Inhaltes in deutscher Sprache, der Umfang beträgt zwischen einer halben und einer ganzen DIN A4-Seite.

Orientieren Sie sich bei der Aufteilung bzw. dem Inhalt Ihrer Zusammenfassung an Kent Becks Artikel: <http://plg.uwaterloo.ca/~migod/research/beck00PSLA.html>.

*We have seen that computer programming is an art,
because it applies accumulated knowledge to the world,
because it requires skill and ingenuity, and especially
because it produces objects of beauty.*

— Donald E. Knuth [1]

Danksagung

Hier können Sie Personen danken, die zum Erfolg der Arbeit beigetragen haben, beispielsweise Ihren Betreuern in der Firma, Ihren Professoren/Dozenten an der htw saar, Freunden, Familie usw.

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation und Zielsetzung	1
1.2	Problemstellung	2
1.3	Ziele der Arbeit	2
1.3.1	Pflichtziele	3
1.3.2	Optionale Erweiterungen	4
1.4	Geplantes Vorgehen	4
1.5	Aufbau der Arbeit	5
2	Grundlage	7
2.1	Grundbegriffe und Definitionen	7
3	Analyse	9
4	Konzeption	11
5	Implementierung	13
6	Evaluation	15
7	Zusammenfassung und Ausblick	17
	Literatur	19
	Abbildungsverzeichnis	21
	Tabellenverzeichnis	21
	Listings	21
	Abkürzungsverzeichnis	23
A	Erster Abschnitt des Anhangs	27

1 Einleitung

1.1 Motivation und Zielsetzung

Die fortschreitende Digitalisierung betrieblicher Abläufe führt dazu, dass die Verfügbarkeit von Netzwerkdiensten und Anwendungen in modernen IT-Umgebungen eine zentrale Voraussetzung für einen stabilen Arbeitsbetrieb darstellt. Bereits kleinere Störungen in Netzwerkverbindungen oder auf Anwendungsebene können dazu führen, dass Arbeitsprozesse unterbrochen, Reaktionszeiten verlängert oder zentrale IT-Dienste nur eingeschränkt nutzbar werden. Vor diesem Hintergrund gewinnt ein systematisches und möglichst automatisiertes Monitoring zunehmend an Bedeutung.

Die Wahl des Themas „Netzwerk-Monitoring Dashboard mit Automatisierung“ ergibt sich aus mehreren fachlichen und praktischen Aspekten.

- **Praxisbezug aus dem Praktikum:** Im Rahmen des IT-Service-Praktikums beim Bildungscampus Saarland wurde deutlich, wie stark Netzwerkstörungen den laufenden Betrieb beeinträchtigen können. Dabei zeigte sich zugleich die Relevanz eines modernen und automatisierten Netzwerk-Monitorings.
- **Berufliche Relevanz:** Die angestrebte fachliche Ausrichtung liegt im Bereich DevOps und Netzwerkinfrastruktur. Monitoring stellt in diesem Umfeld eine zentrale Kompetenz dar und besitzt zugleich eine hohe Bedeutung auf dem aktuellen IT-Arbeitsmarkt.
- **Technisches Interesse:** Das Thema verbindet zentrale Interessensfelder der praktischen Informatik, insbesondere Netzwerkinfrastruktur, Automatisierung sowie moderne Werkzeuge des IT-Betriebs.

Der Praxisbezug des Themas ist besonders relevant, da Monitoring im IT-Service nicht nur der technischen Beobachtung von Systemen dient, sondern unmittelbar zur Sicherstellung betrieblicher Abläufe beiträgt. Werden Netzwerkprobleme oder Leistungsabfälle zu spät erkannt, können Helpdesk-Prozesse, Fernzugriffe, interne Verwaltungsanwendungen oder weitere digitale Dienste verzögert oder gestört werden. Ein automatisiertes Monitoring kann in solchen Situationen dazu beitragen, Störungen früher zu erkennen, Zustände transparenter darzustellen und die Reaktionsfähigkeit des IT-Betriebs zu verbessern.

Darüber hinaus besitzt das Thema auch aus fachlicher Sicht eine hohe Relevanz, da es mehrere Kernbereiche der praktischen Informatik miteinander verbindet. Dazu zählen unter anderem verteilte Systeme, Netzwerkinfrastruktur, automatisierte Datenerfassung, Zeitreihenverarbeitung, Visualisierung und einfache Alarmierungsmechanismen. Die Arbeit greift damit sowohl technische als auch organisatorische Fragestellungen auf und eignet sich daher besonders für eine praxisnahe Abschlussarbeit mit wissenschaftlichem Anspruch.

Das übergeordnete Ziel der Arbeit besteht darin, ein nachvollziehbares und technisch realisierbares Konzept für ein Monitoring-System zu entwickeln und dessen Umsetzbarkeit anhand eines Prototyps zu demonstrieren. Neben der rein technischen Realisierung soll auch gezeigt werden, wie sich verschiedene Werkzeuge sinnvoll kombinieren lassen,

1 Einleitung

um einen konsistenten Monitoring-Prozess von der Datenerfassung bis zur Visualisierung und Alarmierung abzubilden.

1.2 Problemstellung

In modernen IT-Infrastrukturen ist die kontinuierliche Überwachung von Netzwerkverbindungen, insbesondere im LAN- und VPN-Bereich, sowie des Zustands von Anwendungen von zentraler Bedeutung für einen stabilen und zuverlässigen Betrieb. Gerade in IT-Service-Umgebungen können Netzwerkstörungen oder Fehler auf Anwendungsebene in kurzer Zeit zu erheblichen betrieblichen Beeinträchtigungen führen. Dies gilt insbesondere dann, wenn mehrere Systeme voneinander abhängen und Störungen nicht isoliert, sondern kettenartig auf andere Dienste wirken.

In vielen IT-Service-Abteilungen werden Netzwerkprobleme jedoch weiterhin vor allem durch manuelle Ping-Tests oder unregelmäßige Einzelprüfungen erkannt. Solche Verfahren können zwar in einfachen Situationen kurzfristig hilfreich sein, sie sind jedoch nur bedingt geeignet, um größere oder dynamische Infrastrukturen zuverlässig zu überwachen. Die fehlende Automatisierung hat zur Folge, dass Störungen verspätet entdeckt werden, der manuelle Prüfaufwand hoch bleibt, historische Daten für weiterführende Analysen fehlen und bei Ausfällen keine unmittelbare Alarmierung erfolgt.

Diese Vorgehensweise ist nicht nur ineffizient, sondern auch fehleranfällig. Manuelle Kontrollen sind stark vom Zeitpunkt der Durchführung, vom Erfahrungswissen der Beteiligten und von der systematischen Dokumentation einzelner Beobachtungen abhängig. Dadurch entsteht das Risiko, dass kurzzeitige Ausfälle unbemerkt bleiben oder dass Zusammenhänge zwischen verschiedenen Fehlerbildern im Nachhinein nicht mehr zuverlässig rekonstruiert werden können.

Hinzu kommt, dass ohne kontinuierlich gespeicherte Messwerte nur eingeschränkt beurteilt werden kann, ob ein Problem einmalig aufgetreten ist oder ob sich bereits über längere Zeit ein negativer Trend abgezeichnet hat. Für einen professionellen IT-Betrieb ist jedoch nicht nur die unmittelbare Störungserkennung relevant, sondern auch die Möglichkeit, Entwicklungen über einen längeren Zeitraum nachzuvollziehen, Grenzwerte zu definieren und daraus gezielte Maßnahmen abzuleiten.

Ein professioneller IT-Betrieb erfordert daher ein Monitoring, das automatisiert, kontinuierlich und nachvollziehbar arbeitet. Es sollte Metriken auf unterschiedlichen Ebenen erfassen, diese zentral speichern, für operative und analytische Zwecke visualisieren und bei kritischen Zuständen eine zeitnahe Alarmierung ermöglichen. Genau an dieser Stelle setzt die vorliegende Arbeit an, indem ein prototypisches Monitoring-System für Netzwerk- und Anwendungsebene konzipiert und umgesetzt wird.

1.3 Ziele der Arbeit

Ziel der Arbeit ist die Entwicklung eines Monitoringsystems für LAN-/VPN-Verbindungen sowie für eine Spring-Boot-Anwendung. Das System soll Messdaten automatisiert erfassen, speichern, visualisieren und bei kritischen Zuständen eine Alarmierung auslösen. Damit verfolgt die Arbeit das Ziel, einen durchgängigen technischen Ablauf von der Generierung über die Erfassung und Speicherung bis hin zur Auswertung und Benachrichtigung abzubilden.

Die wesentlichen Zielsetzungen lauten wie folgt:

- Erfassung von Anwendungsmetriken durch eine Spring-Boot-Komponente mit Micrometer, insbesondere HTTP-Antwortzeiten, Fehlerquoten, Thread-Anzahl, JVM-

Speicher und CPU-Auslastung.

- Überwachung zusätzlicher Netzwerkmetriken wie Ping-Latenz, Erreichbarkeit, VPN-Status und Port-Verfügbarkeit ausgewählter Dienste.
- Regelmäßige Speicherung aller Metriken in Prometheus als Zeitreihendatenbasis zur Auswertung aktueller und historischer Werte.
- Visualisierung zentraler Kennzahlen aus Anwendung, Netzwerk und Host-Systemen in einem Grafana-Dashboard.
- Umsetzung einer einfachen Alarmierungsfunktion, die bei Ausfällen oder Schwellenwertüberschreitungen automatisiert eine E-Mail-Benachrichtigung versendet.

Die einzelnen Zielsetzungen greifen ineinander. Die Datenerfassung bildet die Grundlage für jede weitere Verarbeitung, da ohne verlässliche Metriken weder eine Zeitreihenspeicherung noch eine aussagekräftige Visualisierung möglich ist. Ebenso setzt eine funktionierende Alarmierung voraus, dass Zustandsänderungen und Grenzwertverletzungen zunächst systematisch erkannt und ausgewertet werden.

Aus fachlicher Sicht verfolgt die Arbeit damit nicht nur die Implementierung einzelner Werkzeuge, sondern die Entwicklung einer durchgängigen Monitoring-Kette. Entscheidend ist dabei, dass die verschiedenen Komponenten nicht isoliert betrachtet, sondern als zusammenhängendes System verstanden werden. Dadurch wird die Arbeit über eine reine Tool-Installation hinaus erweitert und erhält einen eigenständigen konzeptionellen Mehrwert.

Um den Arbeitsumfang innerhalb des vorgegebenen Bearbeitungszeitraums realistisch zu gestalten, wird zwischen Pflichtzielen und optionalen Erweiterungen unterschieden. Diese Trennung ist notwendig, um einen klaren Mindestumfang festzulegen und zugleich weiterführende Entwicklungsmöglichkeiten transparent zu machen.

1.3.1 Pflichtziele

Die Pflichtziele definieren den Kern der Arbeit und müssen im Rahmen des Projekts umgesetzt werden. Sie bilden zugleich die Grundlage für die spätere Bewertung des Prototyps.

- **Datenerfassung:** Entwicklung einer Spring-Boot-API mit Micrometer-Anbindung zur Erfassung von JVM-Metriken wie Speicher, CPU-Auslastung und Threads sowie von HTTP-Metriken wie Antwortzeit und Fehlerquote.
- **Speicherung:** Aufbau einer Prometheus-basierten Zeitreihenspeicherung mit definierten Scraping-Intervallen, beispielsweise 15 Sekunden, um aktuelle und historische Daten auswertbar zu machen.
- **Visualisierung:** Erstellung eines Grafana-Dashboards mit Panels für Ping-Latenz, VPN-Uptime, Port-Status und JVM-Leistung.
- **Alarmierung:** Implementierung eines Bash-Skripts, das bei Schwellenwertüberschreitungen, beispielsweise bei einer Ping-Latenz von über 200 ms oder einem VPN-Ausfall, automatisiert eine E-Mail-Benachrichtigung versendet.

Diese Pflichtziele decken die für das Vorhaben zentralen Funktionsebenen ab. Sie stellen sicher, dass sowohl auf Netzwerk- als auch auf Anwendungsebene relevante Zustände beobachtet werden können und dass die erhobenen Informationen nicht nur gesammelt,

1 Einleitung

sondern auch praktisch nutzbar gemacht werden. Die Kombination aus Datenerhebung, Speicherung, Visualisierung und Alarmierung bildet damit den funktionalen Kern des Prototyps.

1.3.2 Optionale Erweiterungen

Die optionalen Erweiterungen gehen über den definierten Mindestumfang hinaus und dienen dazu, mögliche Ausbaustufen des Systems aufzuzeigen. Sie sind insbesondere dann relevant, wenn einzelne Kernfunktionen früher als geplant umgesetzt werden oder wenn sich im Projektverlauf zusätzlicher Spielraum ergibt.

- **Ansible-Integration:** Automatisierte Bereitstellung der Monitoring-Infrastruktur auf mehreren Servern.
- **Zusätzliche Dashboards:** Getrennte Ansichten für Anwendungs-, Netzwerk- und Host-Ebene.
- **Erweiterte Alarmregeln:** Mehrkriterielle Alarmierung, etwa bei gleichzeitiger Prüfung von VPN-Status, Ping-Latenz und Port-Erreichbarkeit.
- **Rollen- und Rechte management:** Verwaltung von Benutzerrollen und Zugriffsrechten innerhalb des Grafana-Dashboards.

Die optionalen Erweiterungen verdeutlichen, dass das entworfene System grundsätzlich skalierbar und ausbaufähig angelegt ist. Gleichzeitig unterstreichen sie, dass die vorliegende Arbeit einen bewussten Fokus auf einen belastbaren Kernumfang legt, ohne weiterführende Entwicklungsmöglichkeiten auszuschließen.

1.4 Geplantes Vorgehen

Das Vorgehen orientiert sich an einem typischen Ablauf informatischer Abschlussarbeiten. Zunächst wird der fachliche und organisatorische Rahmen präzisiert, insbesondere im Hinblick auf die zu überwachenden Hosts, Netzwerkpfade und VPN-Verbindungen, geeignete Schwellenwerte sowie datenschutzbezogene Anforderungen. Diese Präzisierung ist notwendig, um ein realistisches und konsistentes Zielsystem für die weiteren Projektphasen festzulegen.

Anschließend erfolgt eine Recherche zu Spring Boot, Micrometer, Prometheus und Grafana, um die Eignung dieser Technologien für das geplante Monitoring-Szenario zu bewerten. Dabei geht es nicht nur um eine funktionale Betrachtung einzelner Werkzeuge, sondern auch um deren Zusammenspiel innerhalb einer Gesamtarchitektur. Die Recherche dient damit zugleich als Grundlage für spätere Technologieentscheidungen.

Darauf aufbauend wird ein Systementwurf entwickelt, der Architektur und Datenflüsse der Lösung beschreibt. Dieser Entwurf legt fest, welche Komponenten miteinander kommunizieren, wie Metriken erzeugt und übertragen werden und an welchen Stellen Visualisierung sowie Alarmierung ansetzen. Eine saubere konzeptionelle Vorarbeit ist wesentlich, damit die spätere Implementierung nicht nur funktioniert, sondern auch nachvollziehbar strukturiert ist.

Danach erfolgt die prototypische Implementierung, vorzugsweise in einer containerisierten Umgebung mit Docker. Die Containerisierung erleichtert eine reproduzierbare Bereitstellung der einzelnen Komponenten und unterstützt damit sowohl die technische

Umsetzung als auch die spätere Demonstration des Systems. In diesem Schritt wird geprüft, ob sich die zuvor entworfene Architektur mit den ausgewählten Technologien konsistent umsetzen lässt.

Abschließend wird das System anhand typischer Fehlerszenarien wie erhöhter Ping-Latenz, geschlossenen Ports oder eines VPN-Ausfalls getestet und hinsichtlich Erkennungszeit, Dashboard-Qualität und Alarmierungsfunktion bewertet. Die Evaluation dient nicht nur dazu, die Funktionalität des Prototyps nachzuweisen, sondern auch seine Grenzen sichtbar zu machen und mögliche Ansatzpunkte für spätere Erweiterungen zu identifizieren.

1.5 Aufbau der Arbeit

Nach der Einleitung werden zunächst die für das Verständnis der Arbeit erforderlichen fachlichen und technischen Grundlagen dargestellt. Anschließend folgt die Analyse des Problemraums, der Anforderungen sowie relevanter bestehender Ansätze.

Darauf aufbauend beschreibt die Arbeit die Konzeption des entwickelten Systems und erläutert anschließend dessen prototypische Implementierung. Abschließend erfolgt eine Evaluation der Lösung, bevor die zentralen Ergebnisse zusammengefasst und mögliche Erweiterungen im Ausblick dargestellt werden.

2 Grundlage

2.1 Grundbegriffe und Definitionen

3 Analyse

4 Konzeption

5 Implementierung

6 Evaluation

7 Zusammenfassung und Ausblick

Literatur

- [1] Donald E. Knuth. „Computer Programming as an Art“. In: *Communications of the ACM* 17.12 (1974), S. 667–673.

Abbildungsverzeichnis

Tabellenverzeichnis

Listings

Abkürzungsverzeichnis

Anhang

A Erster Abschnitt des Anhangs

In den Anhang gehören „Hintergrundinformationen“, also weiterführende Information, ausführliche Listings, Graphen, Diagramme oder Tabellen, die den Haupttext mit detaillierten Informationen ergänzen.

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Kolophon

Dieses Dokument wurde mit der L^AT_EX-Vorlage für Abschlussarbeiten an der htw saar im Bereich Informatik/Mechatronik-Sensortechnik erstellt (Version 2.25, August 2024). Die Vorlage wurde von Yves Hary und André Miede entwickelt (mit freundlicher Unterstützung von Thomas Kretschmer, Helmut G. Folz und Martina Lehser). Daten: (F)10.95 – (B)426.79135pt – (H)688.5567pt